Lecture 5
Symbolic Representations: Version Spaces

Armando Solar-Lezama

Symbolic representations

So far we have been generating ASTs 1-by-1 and evaluating them

- Our representations have been either complete or partial ASTs

We want ‘symbols’ that concisely represent large sets of ASTs

- by manipulating those symbols, we can eliminate large sets of ASTs

Key challenges:

- Define the symbols
- Define operations over symbols to efficiently incorporate new evidence
- Query the symbols to extract concrete programs

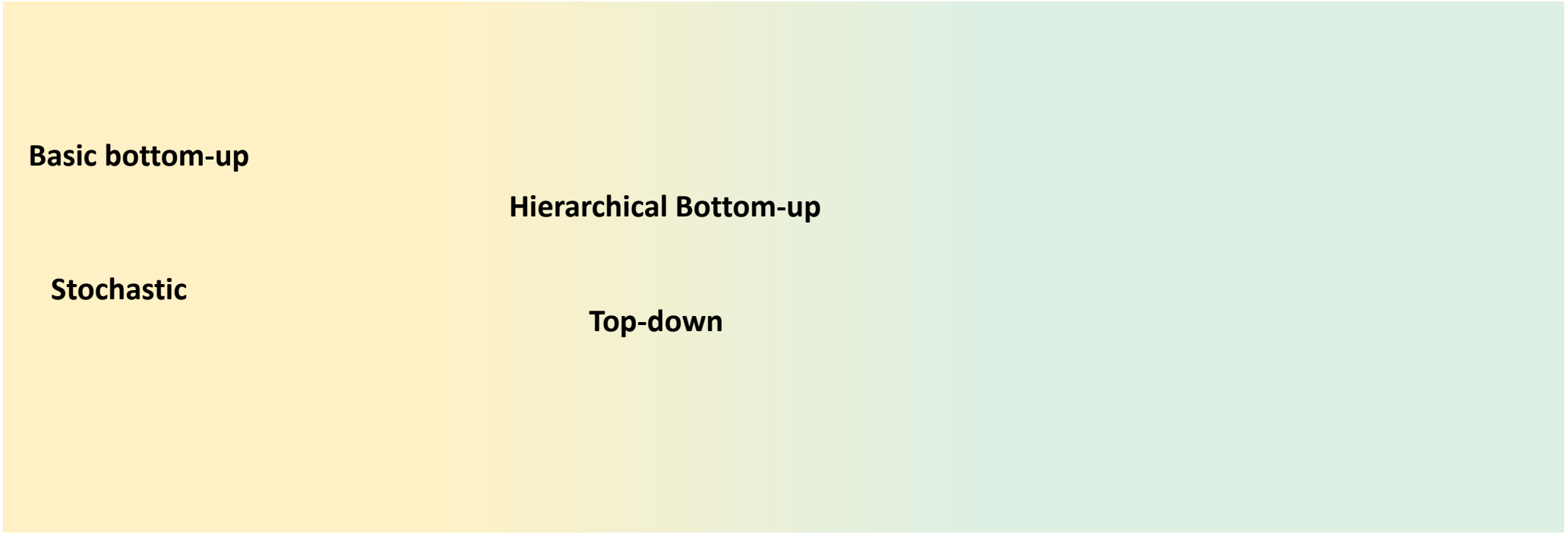
Symbolic vs Enumerative

The boundary between them is not crisp



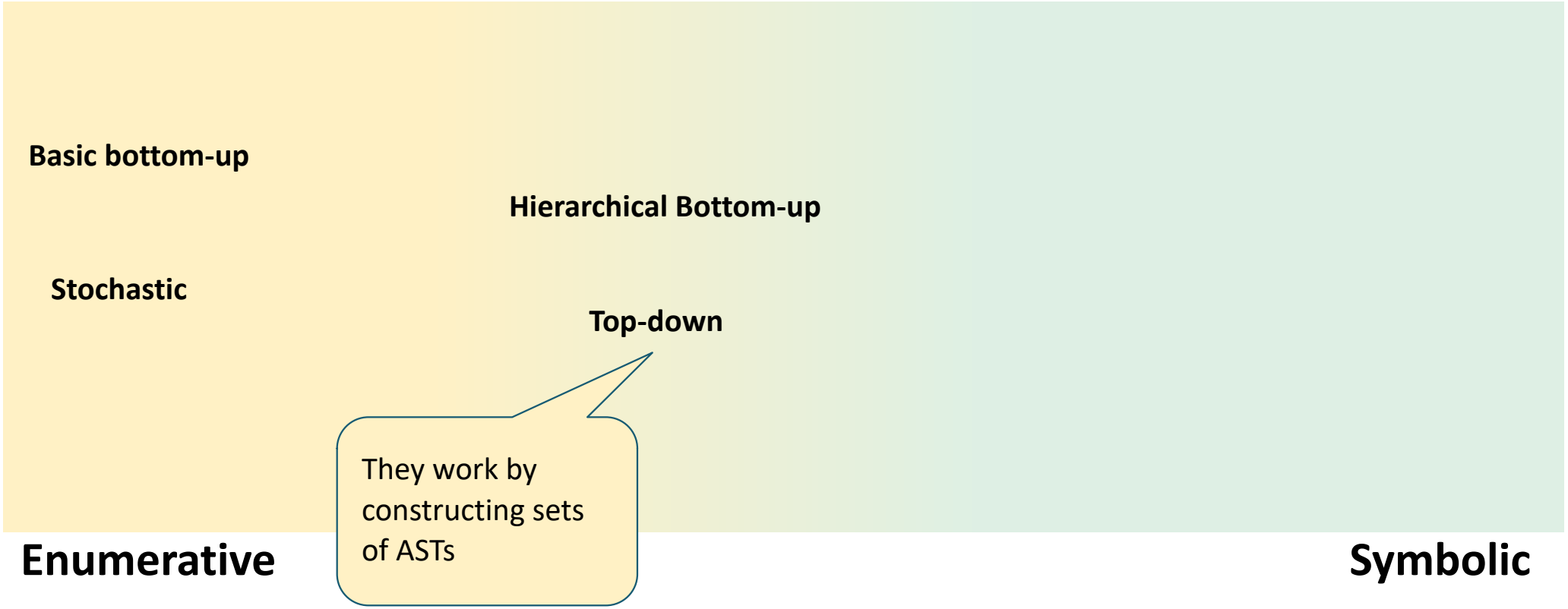
Symbolic vs Enumerative

The boundary between them is not crisp



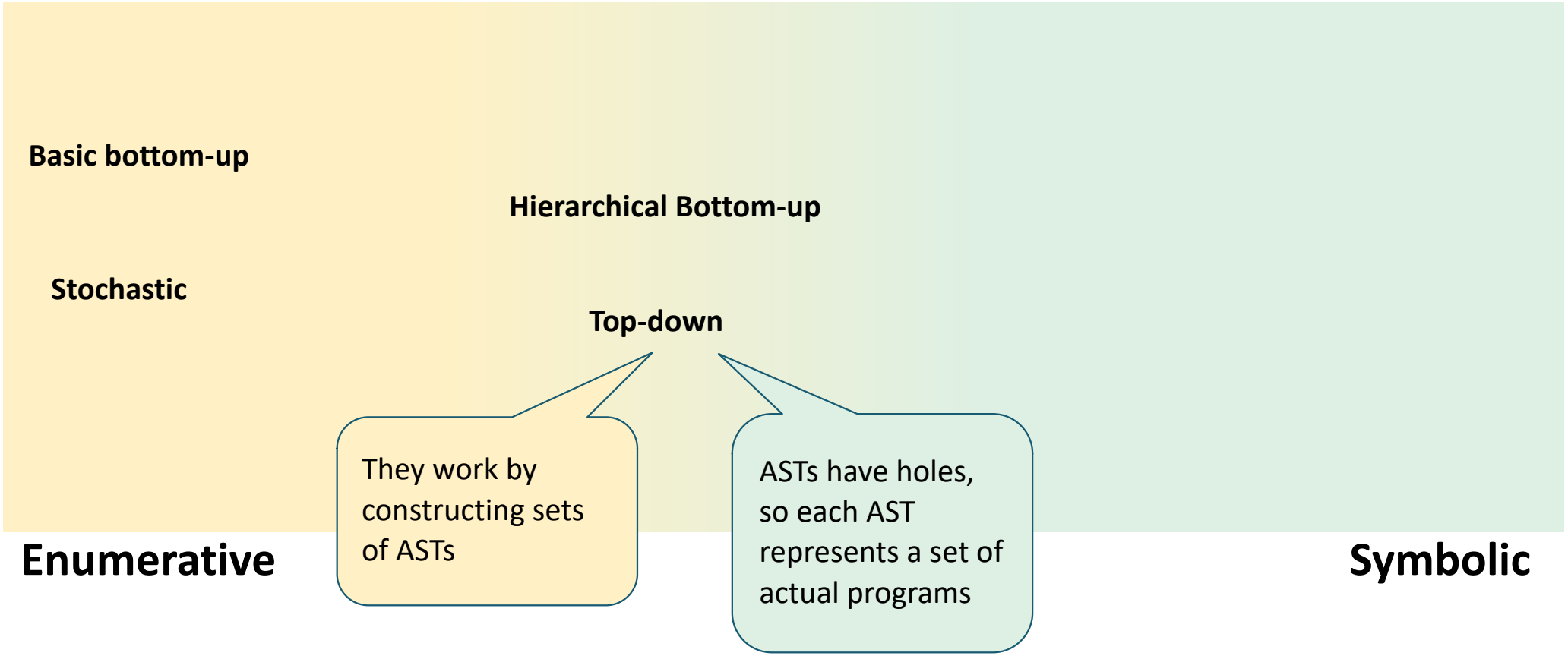
Symbolic vs Enumerative

The boundary between them is not crisp



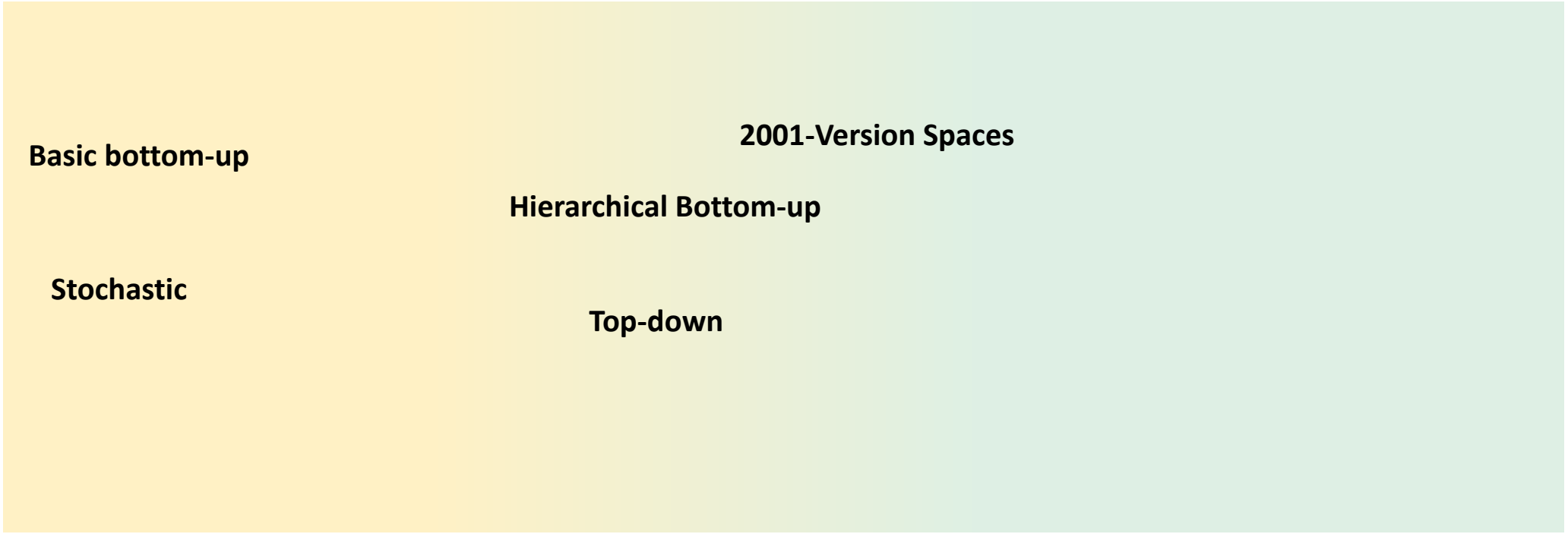
Symbolic vs Enumerative

The boundary between them is not crisp



Symbolic vs Enumerative

The boundary between them is not crisp



Version spaces

Programming by demonstration using version space algebra

Lau, Wolfman, Domingos, Weld, 2001

Version Space Formulation

Hypothesis space H

- Space of possible functions $In \rightarrow Out$

Version Space $VS_{H,D} \subseteq H$

- H is the original hypothesis space
- D is a set of examples i_j, o_j
- $h \in VS_{H,D} \Leftrightarrow \forall\ i, o \in D\ \ h(i) = \mathrm{o}$

Hypothesis space provides *restriction bias*

- Defines what functions one is allowed to consider
- *Preference bias* needs to be provided independently

Partial Ordering of hypothesis

Partial order $h_1 \sqsubseteq h_2$

- h_2 is “better” than h_1

Ex: For boolean hypothesis

- “better” == more general
- $h_1 \sqsubseteq h_2 \Leftrightarrow (h_1 \Rightarrow h_2)$

For booleans, VS forms a lattice

Partial Orders

Set P

Partial order $\leq$ such that $\forall x,y,z \in P$

- $x \leq x$ (reflexive)
- $x \leq y$ and $y \leq x$ implies $x = y$ (asymmetric)
- $x \leq y$ and $y \leq z$ implies $x \leq z$ (transitive)

Can use partial order to define

- Upper and lower bounds
- Least upper bound
- Greatest lower bound

Upper Bounds

If $S \subseteq P$ then

- $x \in P$ is an upper bound of S if $\forall y \in S.\ y \leq x$
- $x \in P$ is the least upper bound of S if
  - x is an upper bound of S, and
  - $x \leq y$ for all upper bounds y of S
- $\vee$ - join, least upper bound, lub, supremum, sup
  - $\vee$ S is the least upper bound of S
  - $x \vee y$ is the least upper bound of {x,y}
- Often written as $\sqcup$ as well

Lower Bounds

If $S \subseteq P$ then

- $x \in P$ is a lower bound of S if $\forall y \in S.\ x \leq y$
- $x \in P$ is the greatest lower bound of S if
  - x is a lower bound of S, and
  - $y \leq x$ for all lower bounds y of S
- $\wedge$ - meet, greatest lower bound, glb, infimum, inf
  - $\wedge$ S is the greatest lower bound of S
  - $x \wedge y$ is the greatest lower bound of {x,y}
- Often written as $\sqcap$ as well

Lattices

If $x \wedge y$ and $x \vee y$ exist for all $x,y \in P$

then P is a lattice

If $\wedge S$ and $\vee S$ exist for all $S \subseteq P$

then P is a complete lattice

All finite lattices are complete

Example of a lattice that is not complete

- Integers I
- For any $x, y \in I$, $x \vee y = \max(x,y)$, $x \wedge y = \min(x,y)$
- But $\vee$ I and $\wedge$ I do not exist
- $I \cup \{+\infty, -\infty\}$ is a complete lattice

Partial Ordering of hypothesis

Partial order $h_1 \sqsubseteq h_2$

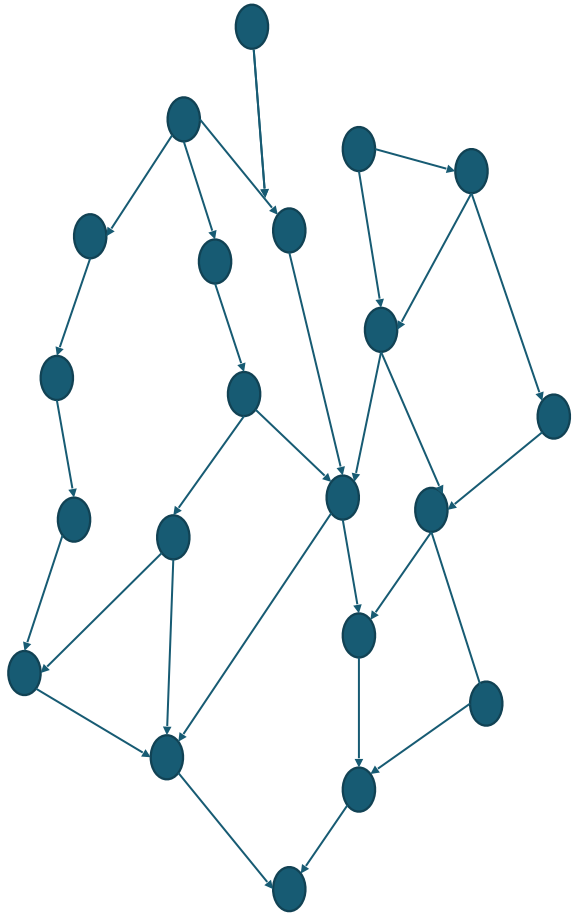
- h_2 is "better" than h_1

Ex: For boolean hypothesis

- "better" == more general
- $h_1 \sqsubseteq h_2 \Leftrightarrow (h_1 \Rightarrow h_2)$

For booleans, VS forms a lattice

- $h_1, h_2 \in VS \Rightarrow h_1 \sqcap h_2 = h_1 \wedge h_2 \in VS$



Most specific hypothesis that satisfies the observations

Boundary set representable

You can represent a VS by the pair (G,S) where

- G is most general hypothesis (i.e. $\top$)
- S is the most specific (i.e. $\bot$)

Applies in general when hypothesis space is partially ordered and version space is a lattice

Update

$$U(VS,\ d) = \{p \in VS \mid p(i) = o\ where\ d = (i, o)\}$$

- Subset of a version space satisfying a new example d

Ex: For boolean HS

- VS=(G,S)
- If $d = (i, true)$

$$U(VS, d) = \left(G, S \vee \lambda x\ .\ if\ x = i\ then\ true\ else\ false\right)$$

- If $d = \left(i, false\right)$

$$U(VS, d) = \left(G \wedge \lambda x\ .\ if\ x = i\ then\ false\ else\ true, S\right)$$

Example: FindSuffix

FS_T: move to the position right before the next occurrence of T.



$FS_{""}$

$FS_{"s"}$

$FS_{"sha"}$

$FS_{"shall"}$

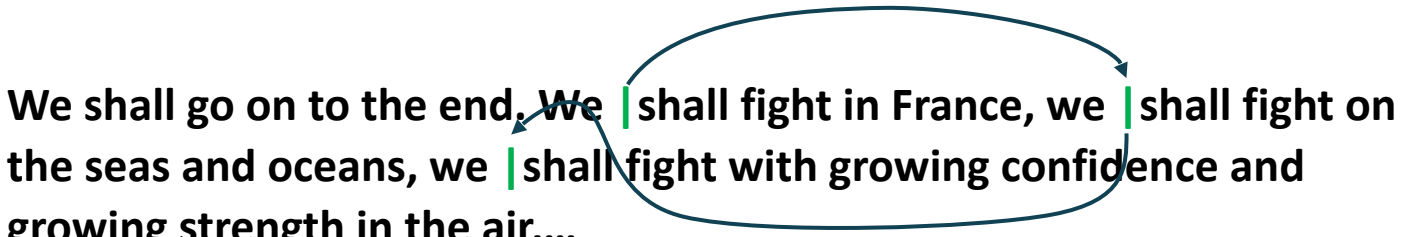
$FS_{"shall\ fight"}$

$FS_{"shall\ fight\ on"}$

$FS_{"shall\ fight\ on\ the\ seas\ and\ oceans,\ we\ shall\ fight\ldots"}$

Example: FindSuffix

FS_T: move to the position right before the next occurrence of T.



$FS_{""}$

$FS_{"s"}$

$FS_{"sha"}$

$FS_{"shall"}$

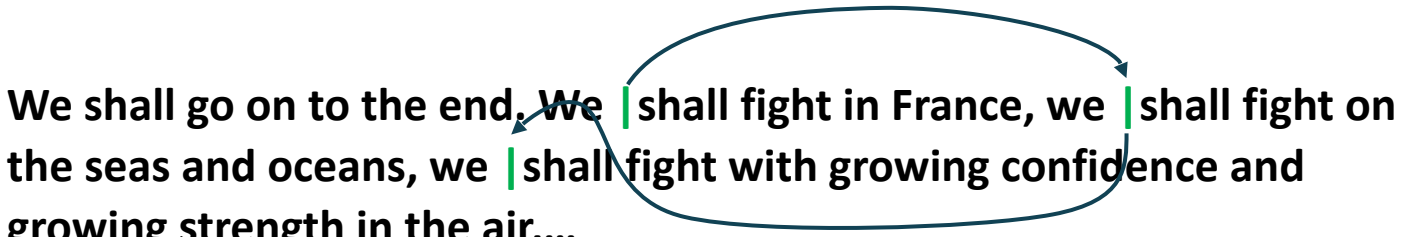
$FS_{"shall\ fight"}$

$FS_{"shall\ fight\ on"}$

$FS_{"shall\ fight\ on\ the\ seas\ and\ oceans,\ we\ shall\ fight\ldots"}$

Example: FindSuffix

FS_T: move to the position right before the next occurrence of T.



$FS_{""}$

$FS_{"s"}$

$FS_{"sha"}$

$FS_{"shall"}$

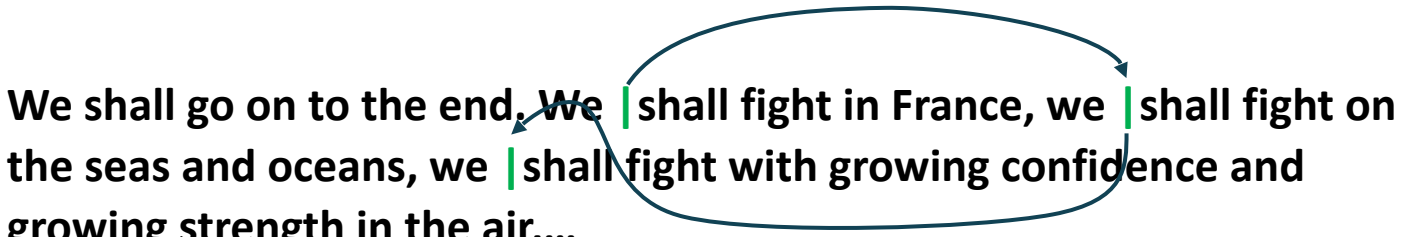
$FS_{"shall\ fight"}$

$FS_{"shall\ fight\ on"}$

$FS_{"shall\ fight\ on\ the\ seas\ and\ oceans,\ we\ shall\ fight\ldots"}$

Example: FindSuffix

FS_T: move to the position right before the next occurrence of T.



$FS_{""}$

$FS_{"s"}$

$FS_{"sha"}$

$FS_{"shall"}$

$FS_{"shall\ fight"}$

$FS_{"shall\ fight\ on"}$

$FS_{"shall\ fight\ on\ the\ seas\ and\ oceans,\ we\ shall\ fight\ldots"}$

Idea

If your hypothesis space is partially ordered and your VS are boundary set representable, you can represent and search very efficiently

If they are not?

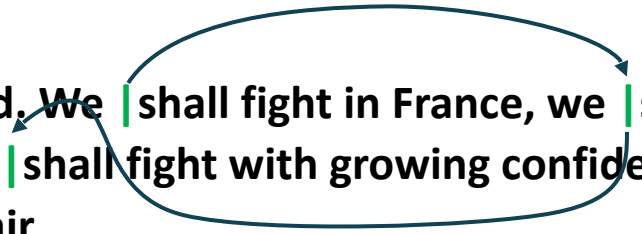
Break them down into simpler hypothesis spaces!

Union

$$VS_{H_1 D} \cup VS_{H_2 D} = VS_{H_1 \cup H_2\ D}$$

FindSuffix U FindPrefix

We shall go on to the end. We | shall fight in France, we | shall fight on the seas and oceans, we | shall fight with growing confidence and growing strength in the air,…

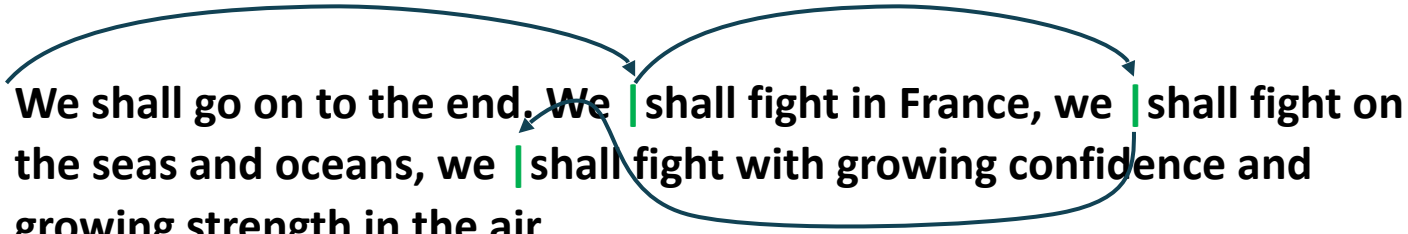


FS(“sh”-”shall fight ”)

U

FP(“we “ – “, we”)

FindSuffix U FindPrefix

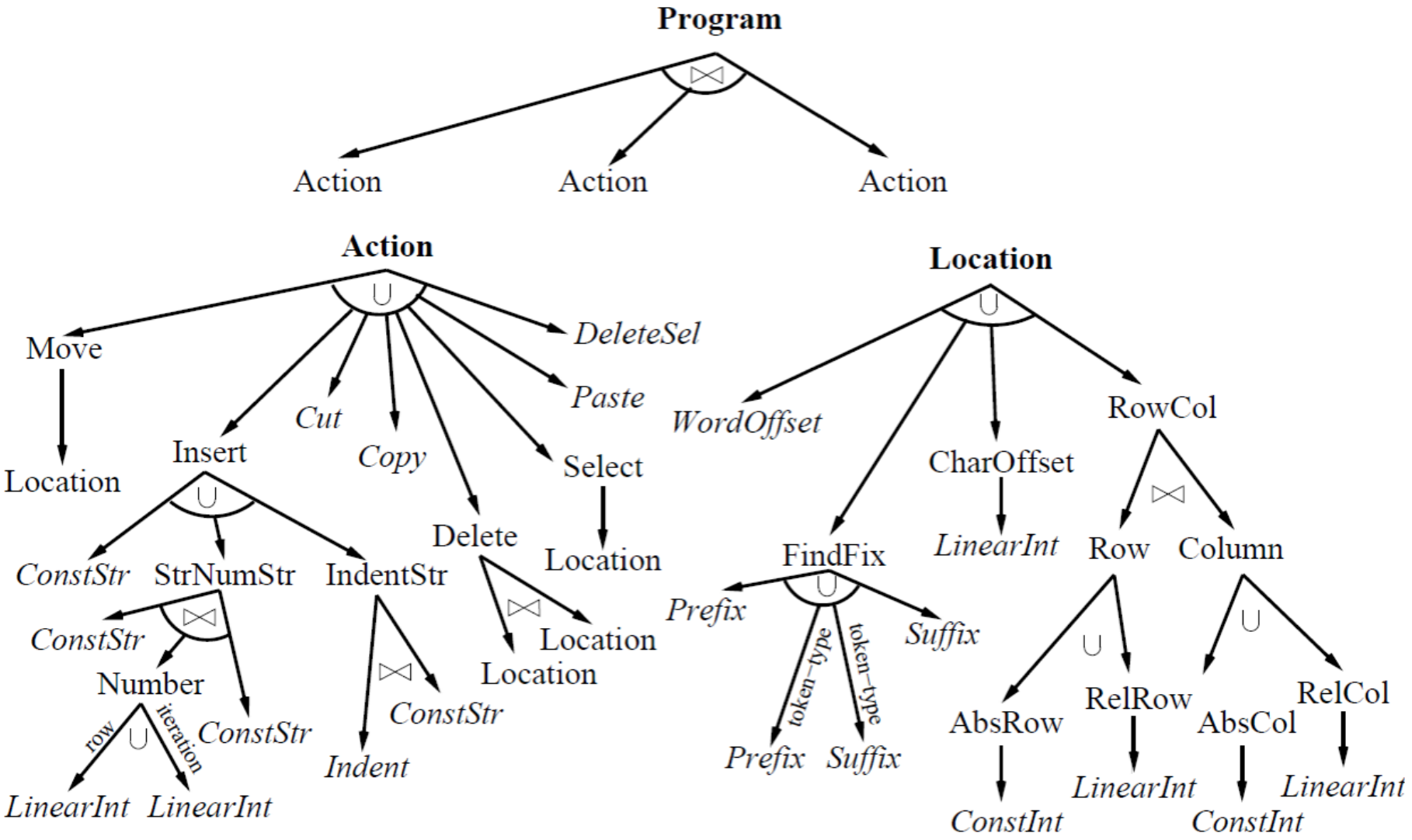


FS(“sh”-”shall fight ”)

U

∅

SMARTedit version space



Flashfill

Spreadsheet Data Manipulations Using Examples

Gulwani, Harris, Singh, CACM August 2012

With slides from Sumit Gulwani and Rishabh Singh

Core language

Trace expr e	:=	$\texttt{Concatenate}(f_1, \ldots, f_n) \mid f$
Atomic expr f	:=	$\texttt{ConstStr}(s) \mid \texttt{SubStr}(v_i, p_1, p_2) \mid \text{Loop}(\lambda w : e)$
Position p	:=	$\texttt{CPos}(k) \mid \text{Pos}(r_1, r_2, c)$
Integer expr c	:=	$k \mid k_1 w + k_2$
Regular expr r	:=	$\texttt{TokenSeq}(T_1, \ldots, T_n) \mid T \mid \varepsilon$

Additional shorthand

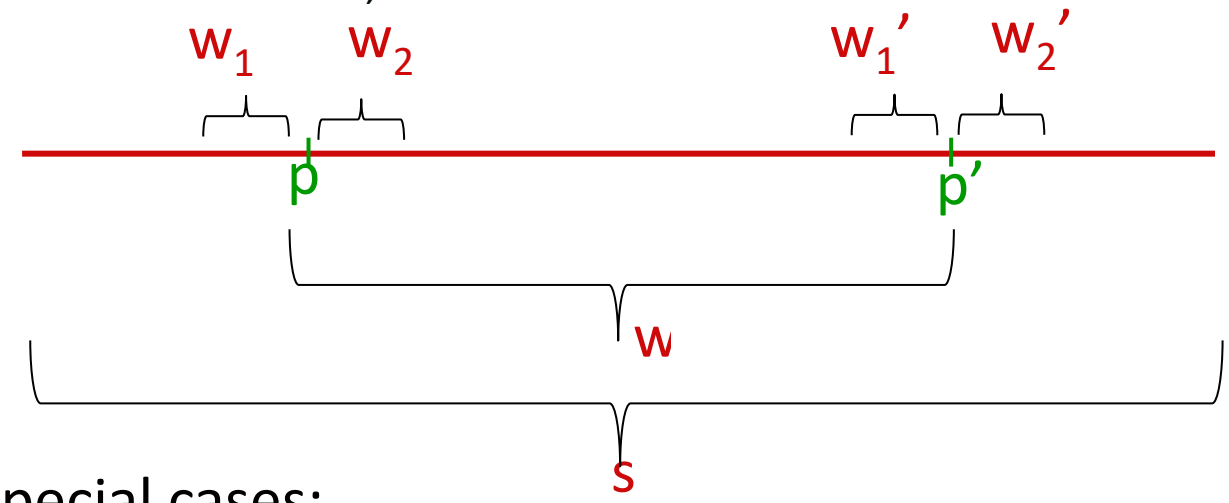
$$SubStr2(vi, r, k) = SubsStr(vi, Pos(\epsilon, r, k), Pos(r, \epsilon, k))$$

- k^{th} occurrence of regular expression r in v_i

Substring Operator

Let w = SubString(s, p, p')

where p = Pos(r_1, r_2, k) and p' = Pos(r_1', r_2', k')



r_1 matches w_1

r_2 matches w_2

r_1' matches w_1'

r_2' matches w_2'

Two special cases:

- $r_1 = r_2' = \epsilon$: This describes the substring
- $r_2 = r_1' = \epsilon$: This describes boundaries around the substring

The general case allows for the combination of the two and is thus a very powerful operator!

Slide by Sumit Gulwani

Additional Control Structure

$$
\begin{array}{rcl}
\text{String program } P & := & \texttt{Switch}\ ((\mathbf{b}_1, \mathrm{e}_1), \ldots, (\mathrm{b}_n, \mathrm{e}_n)) \mid \mathrm{e} \\
\text{Boolean condition b} & := & \mathrm{d}_1 \vee \ldots \vee \mathrm{d}_n \\
\text{Conjunction d} & := & \pi_1 \wedge \ldots \wedge \pi_n \\
\text{Predicate } \pi & := & \texttt{Match}(v_i, \mathrm{r}, k) \mid \neg\, \texttt{Match}(v_i, \mathrm{r}, k)
\end{array}
$$

Syntactic String Transformations: Example

Input v_1	Output
(425)-706-7709	425-706-7709
510.220.5586	510-220-5586
235 7654	425-235-7654
745-8139	425-745-8139

$Switch((b1,\ e1),\ (b2,\ e2))$, where

$b1 \equiv Match(v1, NumTok, 3), \qquad b2 \equiv \neg Match(v1, NumTok, 3),$

$$e1 \equiv Concatenate\begin{pmatrix} SubStr2(v1, NumTok, 1), \\ ConstStr(" - "), \\ SubStr2(v1, NumTok, 2), \\ ConstStr(" - "), \\ SubStr2(v1, NumTok, 3) \end{pmatrix} \quad e_2 \equiv Concatenate\begin{pmatrix} \text{ConstStr}("425-"), \\ ConstStr(" - "), \\ SubStr2(v1, NumTok, 1), \\ ConstStr(" - "), \\ SubStr2(v1, NumTok, 2) \end{pmatrix}$$

Slide by Sumit Gulwani

How does it work

Reuse ideas from version space algebra

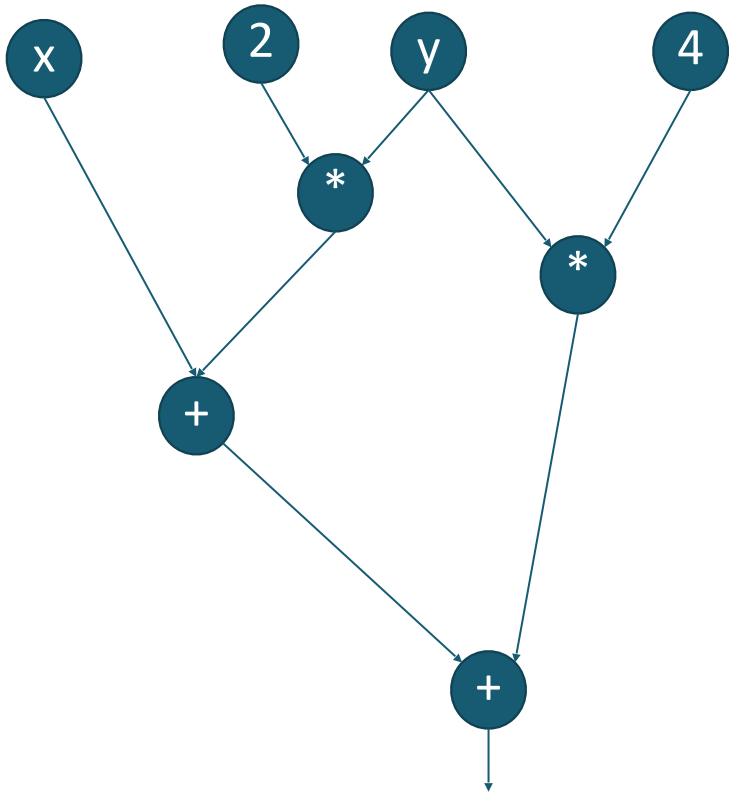
- Start with simple version spaces
- Define combinators to construct complex version spaces from simple ones

New twist:

- Move beyond simple lattice representations

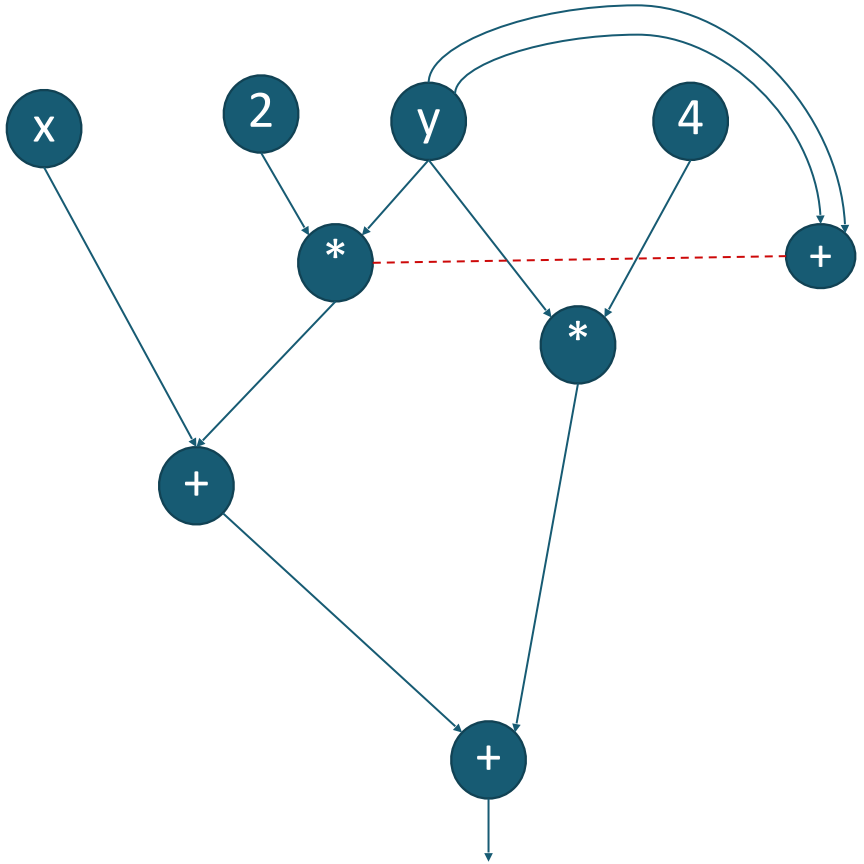
E-Graph

(x+2*y)+4*y



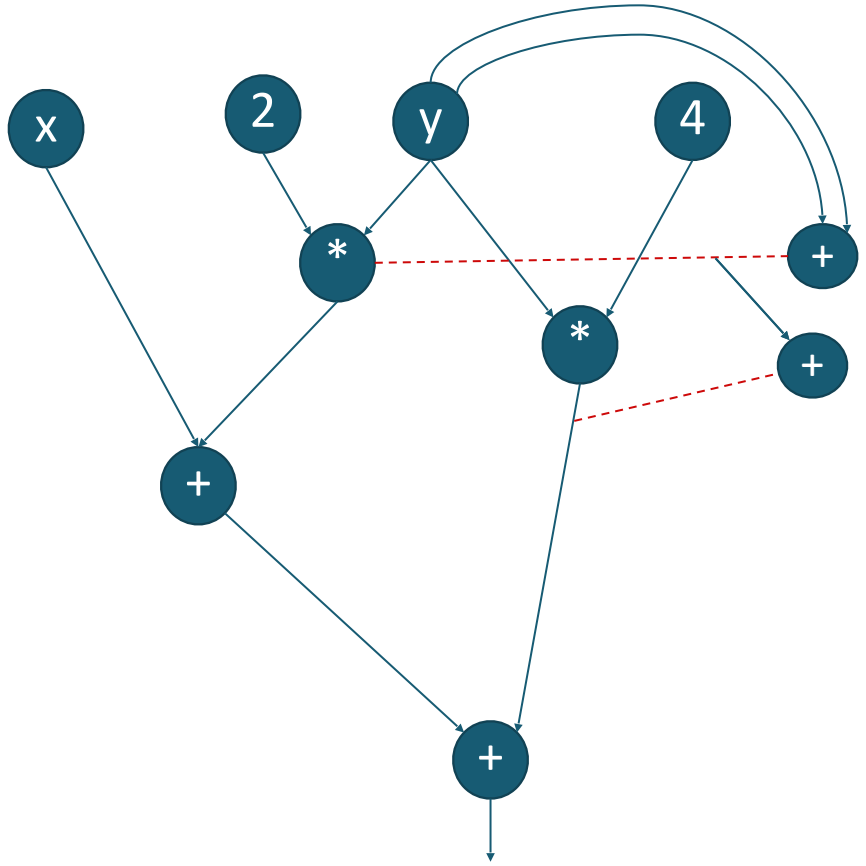
E-Graph

(x+2*y)+4*y



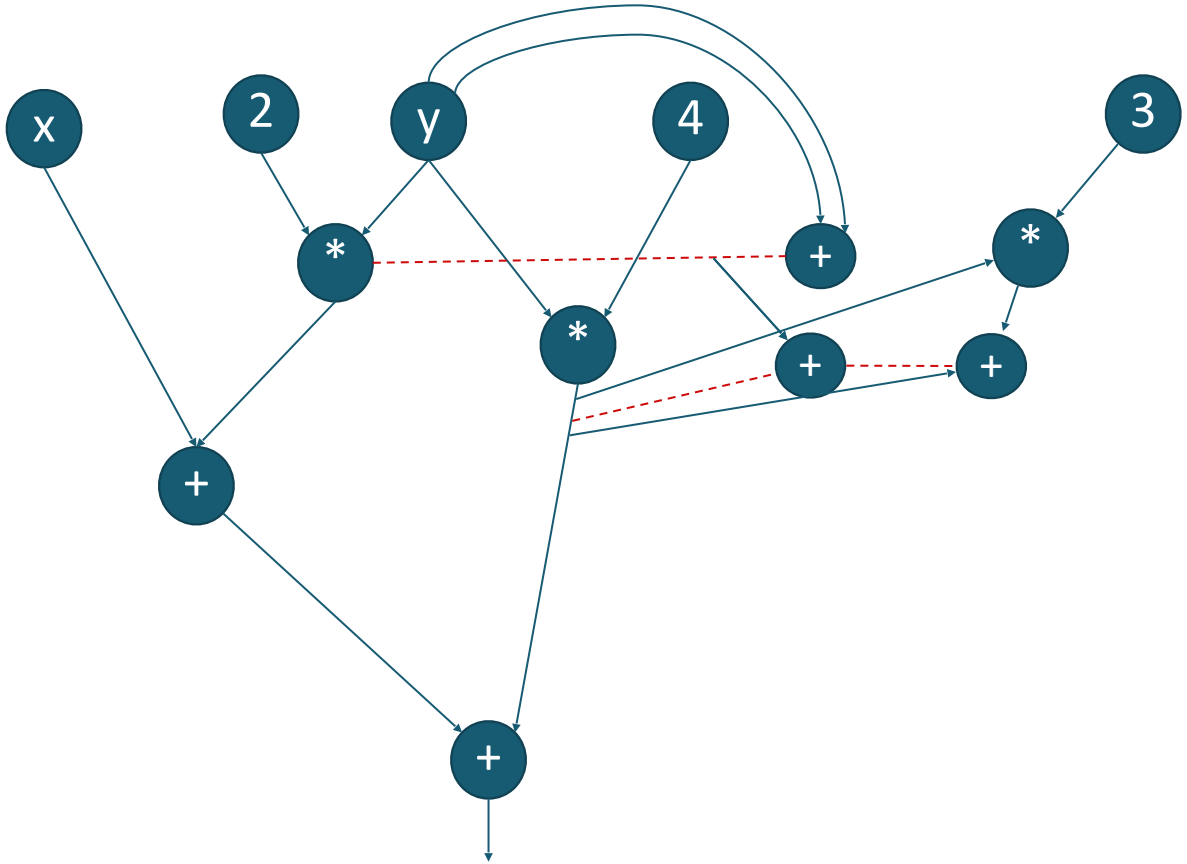
E-Graph

(x+2*y)+4*y



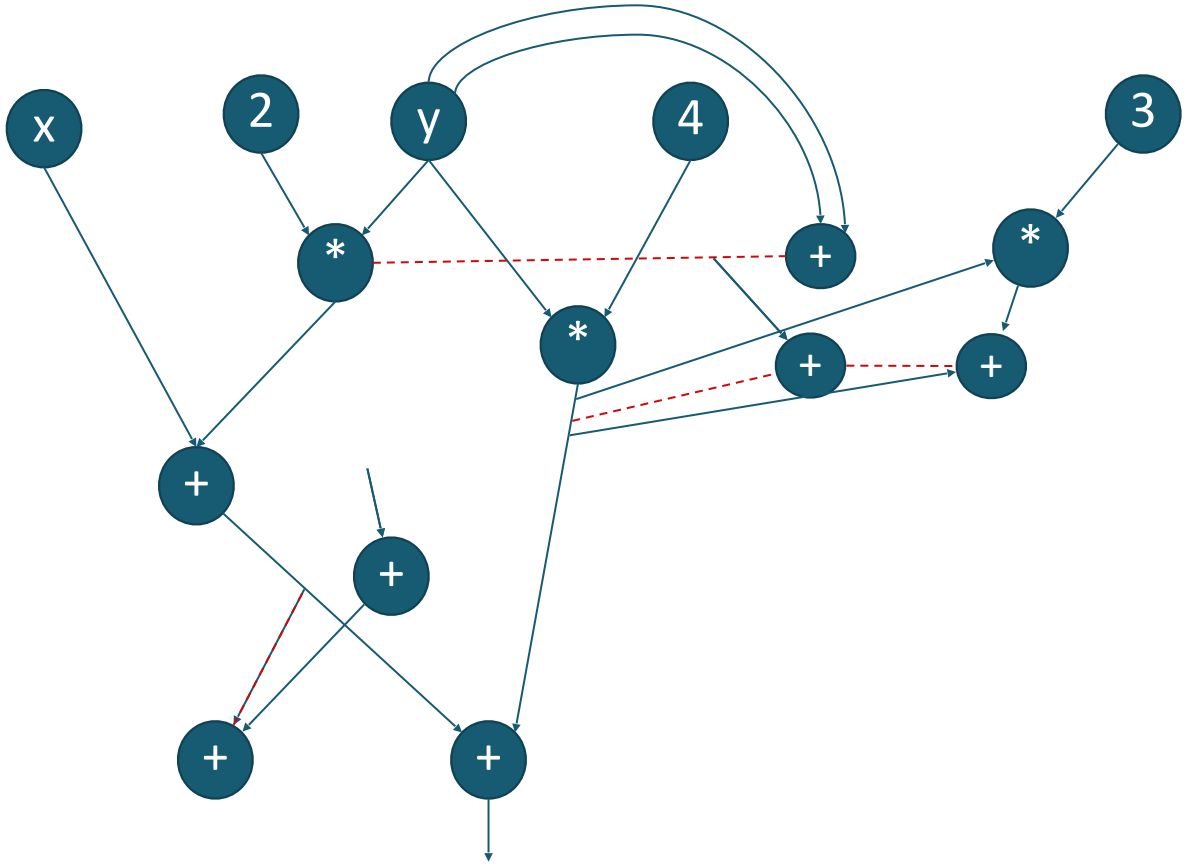
E-Graph

(x+2*y)+4*y



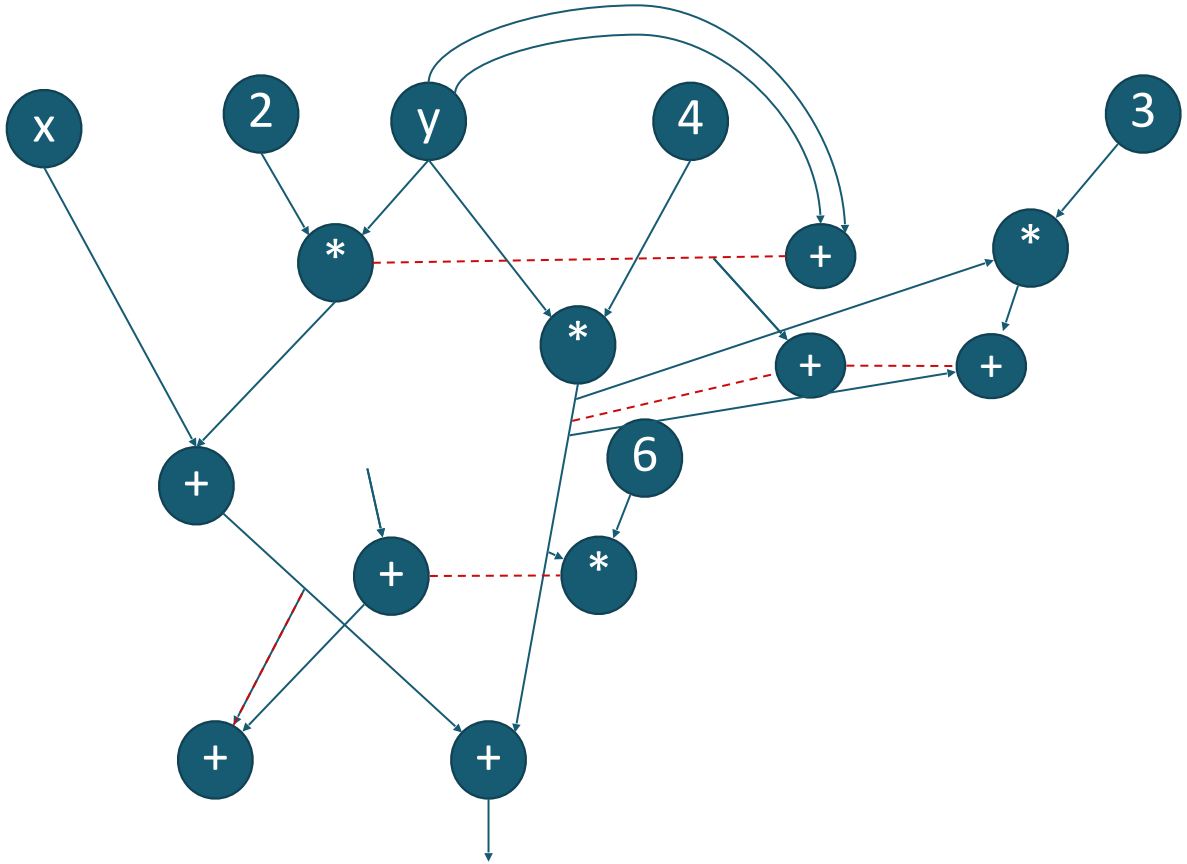
E-Graph

(x+2*y)+4*y



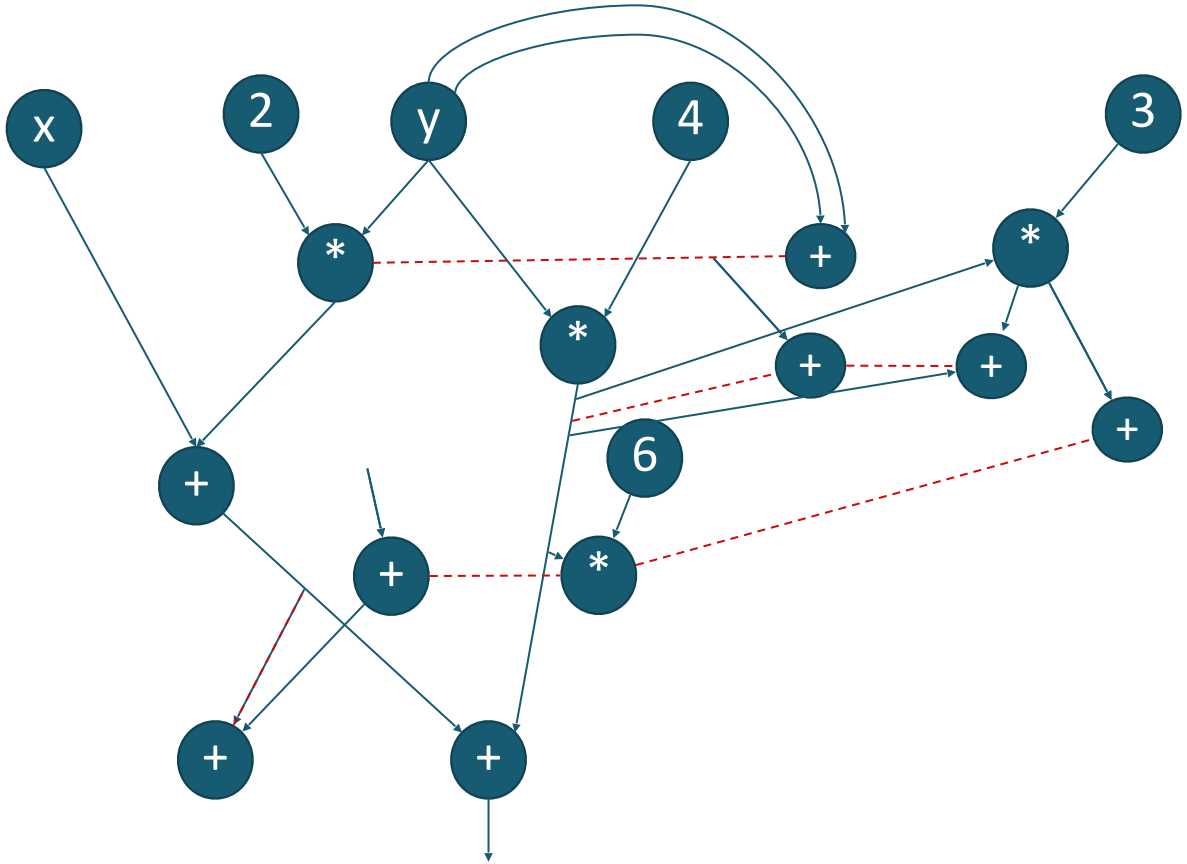
E-Graph

(x+2*y)+4*y



E-Graph

(x+2*y)+4*y



Guarded Expressions

D is a set of examples i_j, o_j H corresponds to trace expressions

Compute $S_j = VS_{(i_j o_j) H}$ and $S = VS_{DH} = \cap S_j$

Solution $= S \cup Switch(\left(p_i,\ VS_{D_i H}\right))$

- Either you can find a solution to D in H, or you can split D and find solutions for subsets $D_i \subset D$
- Partition into subsets D_i must satisfy the following properties:
  - $VS_{D_i H} \neq \emptyset$
  - Partition is minimial
  - You can learn predicates p_i to create the partitions

Learning Trace Expressions

Full Name	Title Name
Rob Miller	Dr. Rob
Saman Amarasinghe	
Sumit Gulwani	
Armando Solar-Lezama	
Martin Rinard	

Slide by Rishabh Singh

Learning Trace Expressions

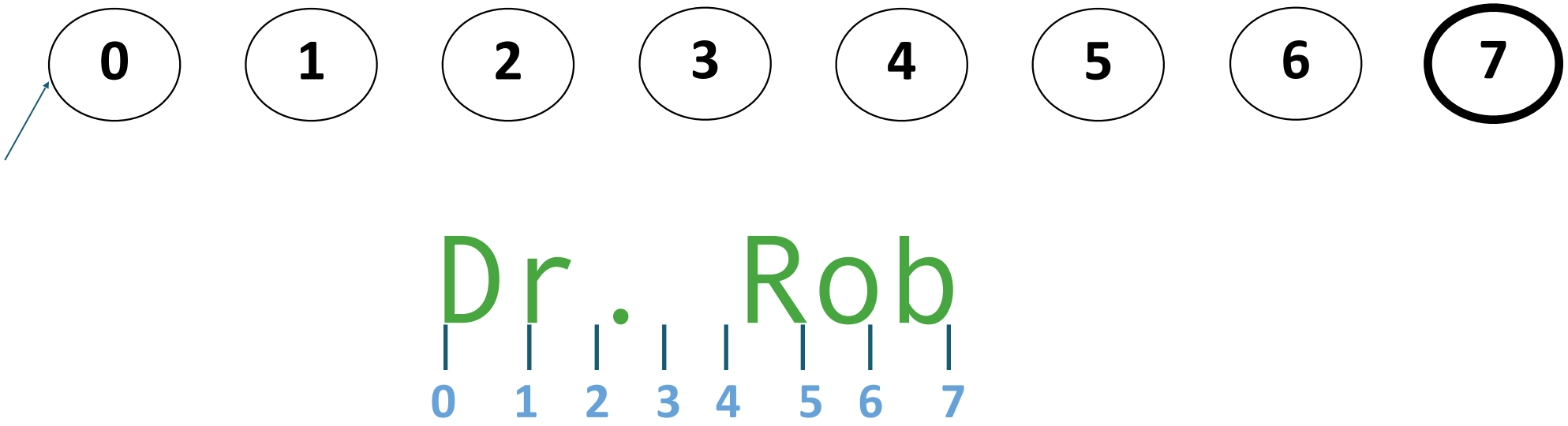
Top level is always concatenation

Concat Expression (Associative)

Rob Miller → Dr. Rob

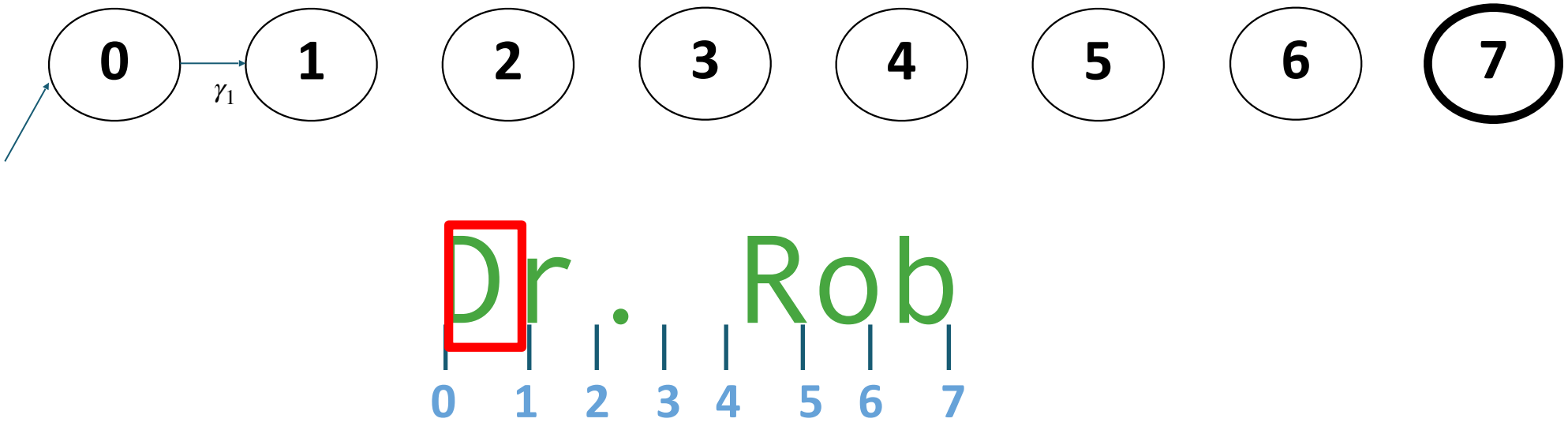
Slide by Rishabh Singh

Concat Expression (Associative)



Slide by Rishabh Singh

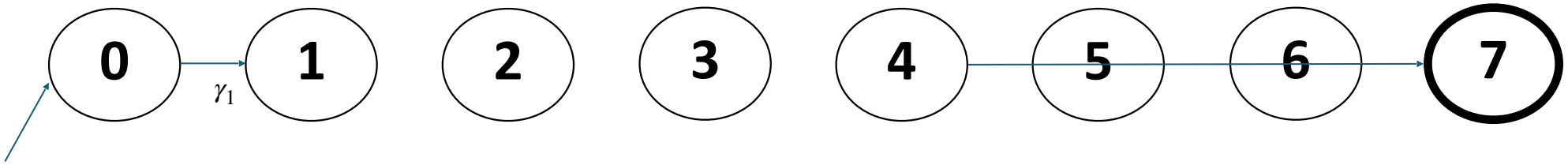
Concat Expression (Associative)

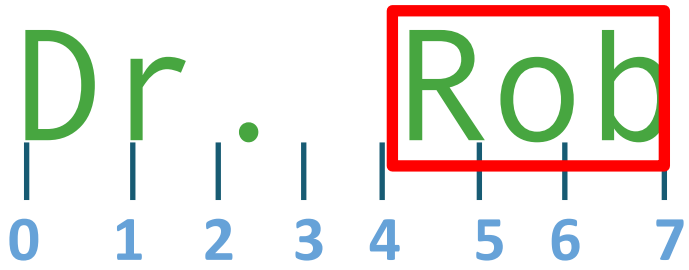


Slide by Rishabh Singh

Concat Expression (Associative)

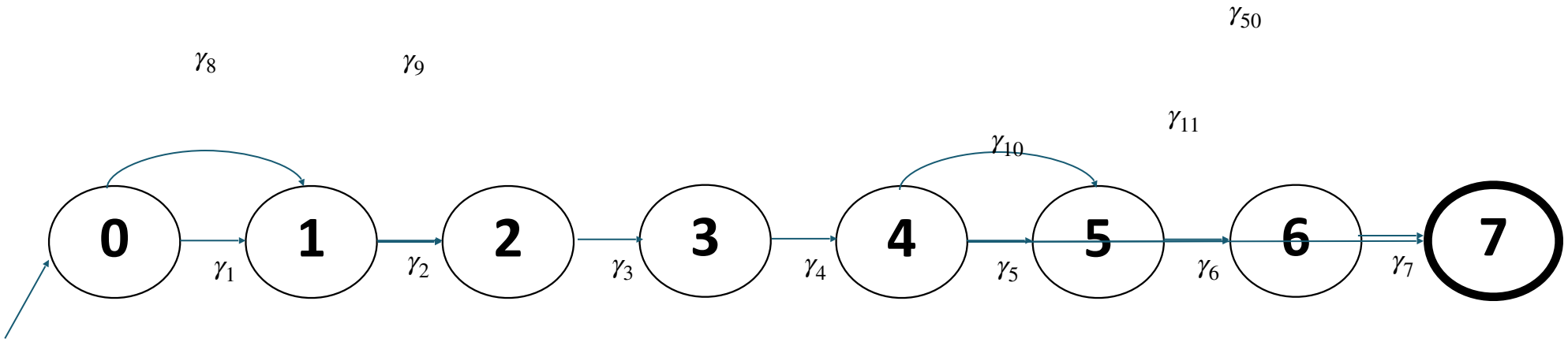
γ_{50}





Slide by Rishabh Singh

Concat Expression (Associative)

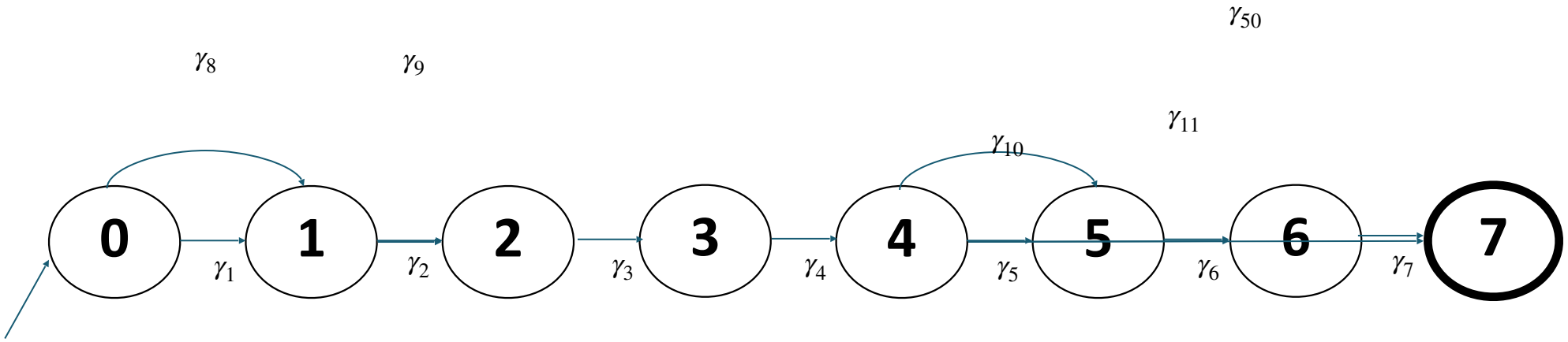


Dr. Rob

0 1 2 3 4 5 6 7

Slide by Rishabh Singh

DAG-based Sharing



Any Path is a valid program

$\gamma_1 \cdot \gamma_2 \cdot \gamma_3 \cdot \gamma_4 \cdot \gamma_5 \cdot \gamma_6 \cdot \gamma_7$

$\gamma_8 \cdot \gamma_2 \cdot \gamma_3 \cdot \gamma_4 \cdot \gamma_{50}$

$\gamma_8 \cdot \gamma_2 \cdot \gamma_3 \cdot \gamma_4 \cdot \gamma_5 \cdot \gamma_6 \cdot \gamma_7$

Exponential Number of paths

Slide by Rishabh Singh

Learning atomic expressions

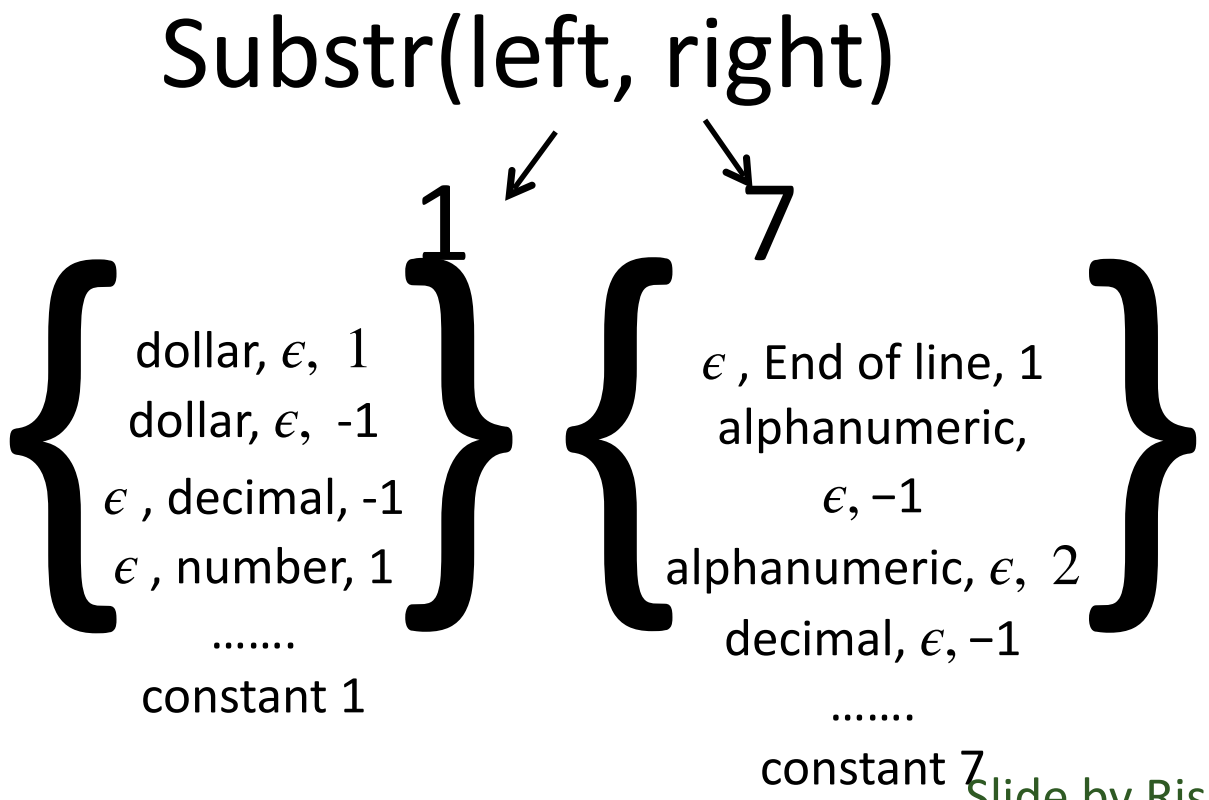
Can only be constants, substring or loop expressions

Constants are trivial to learn

Substring can be factored!

Substring Expression

$145.67 →
1145.677



Slide by Rishabh Singh

Intersection

Armando Solar

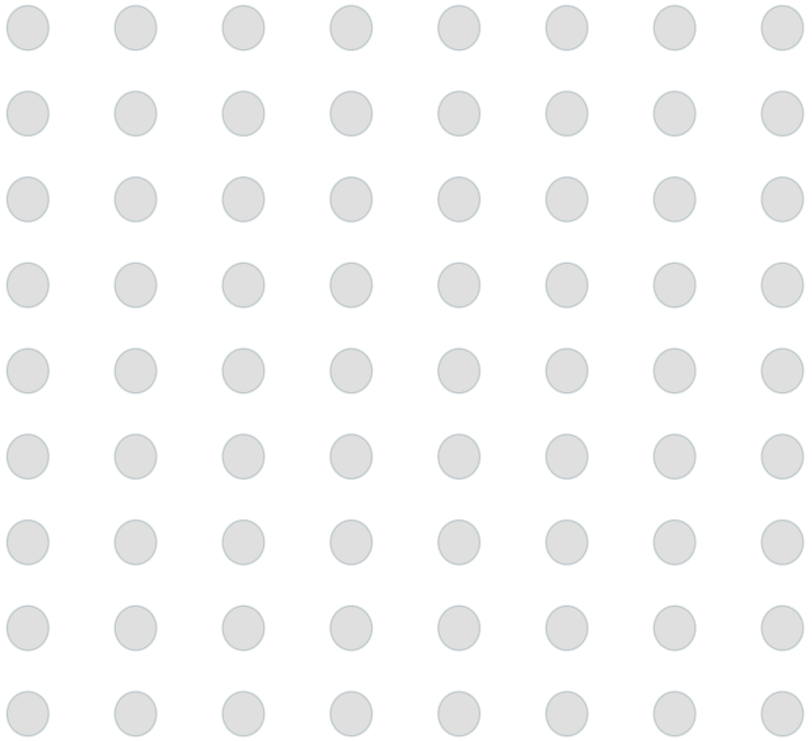
A. Solar



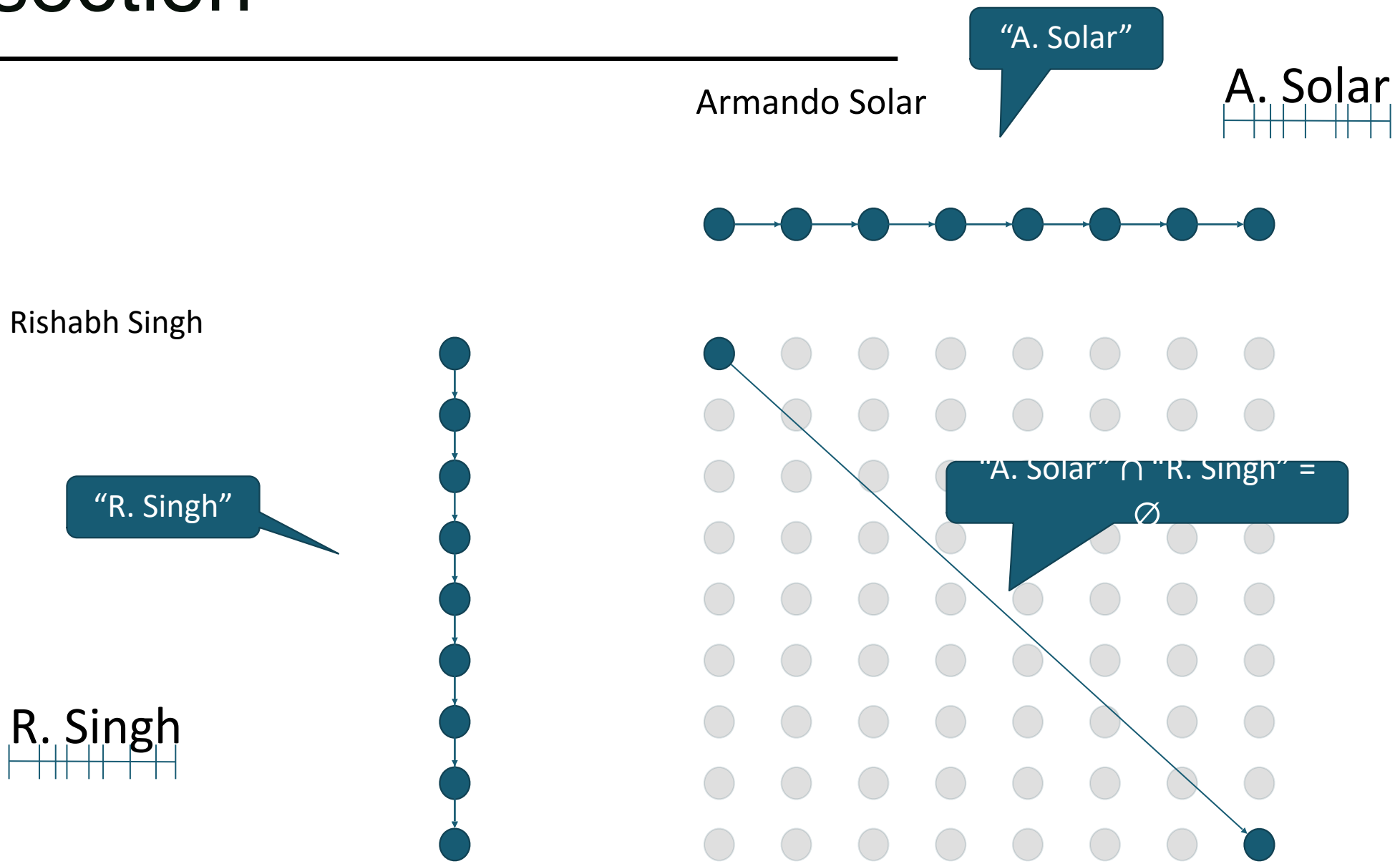
Rishabh Singh



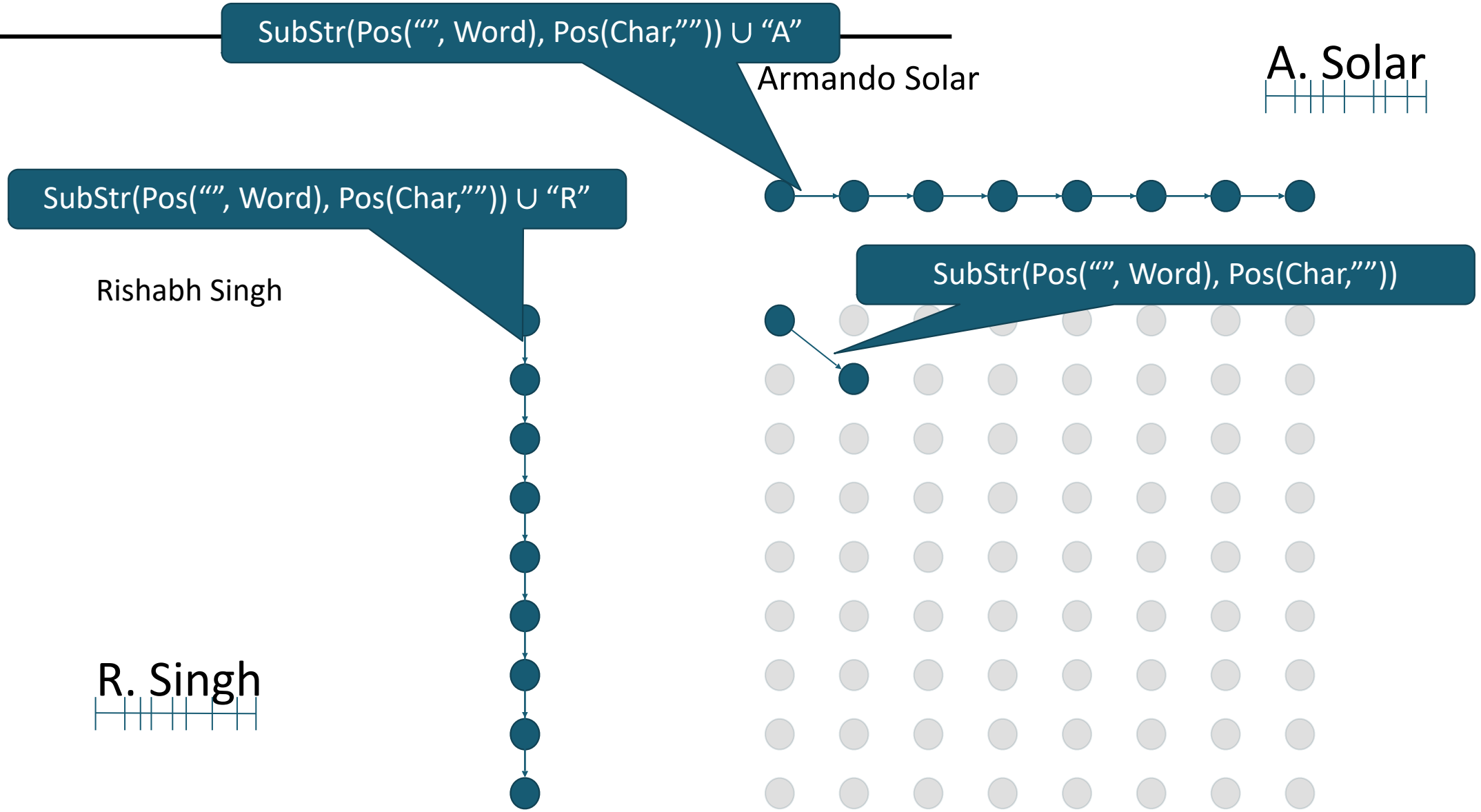
R. Singh



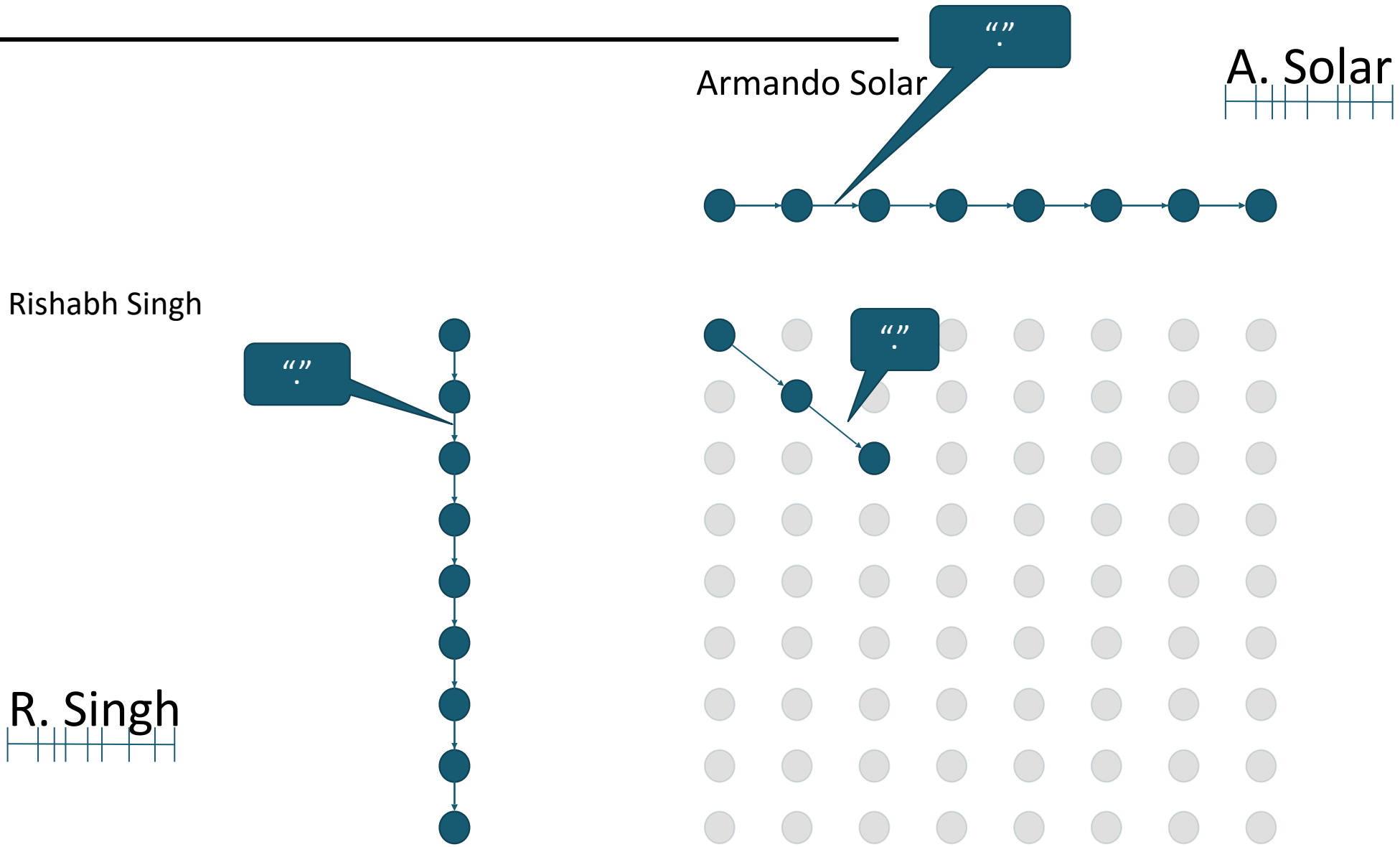
Intersection



Intersection



Intersection



Ranking

Prefer shorter programs.

- Fewer number of conditionals.
- Shorter string expression, regular expressions.

Prefer programs with fewer constants.

Strategies

Baseline: Pick any minimal sized program using minimal number of constants.

Manual: Break conflicts using a weighted score of program features.

Machine Learning: Weights are learned from training data.

Adapted from slide by Sumit Gulwani